

PRACTICAL OSSP

SESSION 6:

TASK 1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create the SERVER

```
koumudi@Koumudi: ~/OSSP/ x + v
GNU nano 7.2 fifo_serv
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <errno.h>

#define SERVER_FIFO "server_fifo"
#define BUFFER_SIZE 1024

int main()
{
    char buffer[BUFFER_SIZE];

    // Create the server FIFO
    if (mkfifo(SERVER_FIFO, 0666) == -1 && errno != EEXIST)
    {
        perror("mkfifo");
        return 1;
    }

    printf("FIFO Server started.\n");
    printf("Waiting for clients...\n");

    while (1)
```

```
koumudi@Koumudi: ~/OSSP/ x + v
GNU nano 7.2
while (1)
{
    int server_fd = open(SERVER_FIFO, O_RDONLY);

    if (server_fd == -1)
    {
        perror("open server FIFO");
        continue;
    }

    ssize_t bytes_read = read(server_fd, buffer, sizeof(buffer) - 1);
    close(server_fd);

    if (bytes_read > 0)
    {
        buffer[bytes_read] = '\0';
        printf("\nMessage received: %s\n", buffer);

        char *separator = strchr(buffer, '|');

        if (separator != NULL)
        {
            *separator = '\0';

            char *client_fifo = buffer;
            char *message = separator + 1;

            printf("Client FIFO: %s\n", client_fifo);
            printf("Processing message: %s\n", message);

            char response[BUFFER_SIZE];

            snprintf(
                response,
```

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2
printf("Client FIFO: %s\n", client_fifo);
printf("Processing message: %s\n", message);

char response[BUFFER_SIZE];

snprintf(
    response,
    sizeof(response),
    "Server received your message: %s",
    message
);

int client_fd = open(client_fifo, O_WRONLY);

if (client_fd == -1)
{
    perror("open client FIFO");
    continue;
}

write(client_fd, response, strlen(response) + 1);

close(client_fd);

printf("Response sent to client.\n");
}
else
{
    printf("Invalid message format.\n");
}
}
}

return 0;
}
```

Create the CLIENT

```
koumudi@Koumudi: ~/OSSP/ × + v
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define SERVER_FIFO "server_fifo"
#define BUFFER_SIZE 1024

int main()
{
    char message[BUFFER_SIZE];
    char client_fifo[BUFFER_SIZE];
    char request[BUFFER_SIZE * 2];
    char response[BUFFER_SIZE];

    // Create a unique FIFO for this client
    snprintf(
        client_fifo,
        sizeof(client_fifo),
        "client_%d_fifo",
        getpid()
    );

    if (mkfifo(client_fifo, 0666) == -1)
    {
        perror("mkfifo client");
        return 1;
    }

    printf("Client started.\n");
    printf("Enter message: ");

    fgets(message, sizeof(message), stdin);
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
fgets(message, sizeof(message), stdin);

message[strcspn(message, "\n")] = '\0';

// Send client FIFO name and message to server
snprintf(
    request,
    sizeof(request),
    "%s%s",
    client_fifo,
    message
);

int server_fd = open(SERVER_FIFO, O_WRONLY);

if (server_fd == -1)
{
    perror("open server FIFO");
    unlink(client_fifo);
    return 1;
}

write(server_fd, request, strlen(request) + 1);

close(server_fd);

// Wait for server response
int client_fd = open(client_fifo, O_RDONLY);

if (client_fd == -1)
{
    perror("open client FIFO");
    unlink(client_fifo);
    return 1;
}
```

```
koumudi@Koumudi: ~/OSSP/ × + ▾
GNU nano 7.2
write(server_fd, request, strlen(request) + 1);

close(server_fd);

// Wait for server response
int client_fd = open(client_fifo, O_RDONLY);

if (client_fd == -1)
{
    perror("open client FIFO");
    unlink(client_fifo);
    return 1;
}

ssize_t bytes_read = read(
    client_fd,
    response,
    sizeof(response) - 1
);

close(client_fd);

if (bytes_read > 0)
{
    response[bytes_read] = '\0';

    printf("Server response: %s\n", response);
}

unlink(client_fifo);

printf("Client finished.\n");

return 0;
}
```

```
koumudi@Koumudi: ~/OSSP/ × + v
koumudi@Koumudi:~$ cd ~/OSSP/PRACTICAL
koumudi@Koumudi:~/OSSP/PRACTICAL$ ls
Session_1 Session_2 Session_3 Session_4 Session_5
koumudi@Koumudi:~/OSSP/PRACTICAL$ mkdir Session_6
koumudi@Koumudi:~/OSSP/PRACTICAL$ cd Session_6
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_6$ nano fifo_server.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_6$ nano fifo_client.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_6$ gcc fifo_server.c -o fifo_server
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_6$ gcc fifo_client.c -o fifo_client
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_6$ ls
fifo_client fifo_client.c fifo_server fifo_server.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_6$ ./fifo_server
FIFO Server started.
Waiting for clients...

Message received: client_1238_fifo|Hello Server
Client FIFO: client_1238_fifo
Processing message: Hello Server
Response sent to client.
```

```
koumudi@Koumudi: ~/OSSP/ × + v
koumudi@Koumudi:~$ cd ~/OSSP/PRACTICAL/Session_6
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_6$ ./fifo_client
Client started.
Enter message: Hello Server
Server response: Server received your message: Hello Server
Client finished.
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_6$ |
```

TASK 2: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

volatile sig_atomic_t signal_received = 0;

void handle_signal(int signal_number)
{
    if (signal_number == SIGINT)
    {
        printf("\nSIGINT received (Ctrl+C).\n");
        signal_received = SIGINT;
    }
    else if (signal_number == SIGTERM)
    {
        printf("\nSIGTERM received.\n");
        signal_received = SIGTERM;
    }
    else if (signal_number == SIGUSR1)
    {
        printf("\nSIGUSR1 received.\n");
        signal_received = SIGUSR1;
    }
}

int main()
{
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);

    printf("POSIX Signal Handler Program\n");
    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");
    printf("Press Ctrl+C to send SIGINT.\n");
```

```
koumudi@Koumudi: ~/OSSP/ × + ∨
GNU nano 7.2
signal(SIGTERM, handle_signal);
signal(SIGUSR1, handle_signal);

printf("POSIX Signal Handler Program\n");
printf("Process ID: %d\n", getpid());
printf("Waiting for signals...\n");
printf("Press Ctrl+C to send SIGINT.\n");
printf("Use another terminal to send SIGTERM or SIGUSR1.\n");

while (1)
{
    pause();

    if (signal_received == SIGINT)
    {
        printf("Handling SIGINT asynchronously...\n");
        signal_received = 0;
    }
    else if (signal_received == SIGTERM)
    {
        printf("Handling SIGTERM asynchronously...\n");
        printf("Terminating program...\n");
        break;
    }
    else if (signal_received == SIGUSR1)
    {
        printf("Handling SIGUSR1 asynchronously...\n");
        signal_received = 0;
    }
}

printf("Program terminated successfully.\n");

return 0;
```

```
koumudi@Koumudi: ~ × + ∨
koumudi@Koumudi:~$ kill -SIGTERM 516
koumudi@Koumudi:~$ |
```

```
koumudi@Koumudi: ~ × + ∨
koumudi@Koumudi:~$ kill -SIGUSR1 YOUR_PID
-bash: kill: YOUR_PID: arguments must be process or job IDs
koumudi@Koumudi:~$ kill -SIGUSR1 516
koumudi@Koumudi:~$ |
```

koumudi@Koumudi: ~/OSSP/ × + ▾

```
koumudi@Koumudi:~$ cd ~/OSSP/PRACTICAL/Session_6
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_6$ ls
fifo_client  fifo_client.c  fifo_server  fifo_server.c  server_fifo
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_6$ nano signal_handler.c
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_6$ gcc signal_handler.c -o signal_handler
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_6$ ./signal_handler
POSIX Signal Handler Program
Process ID: 516
Waiting for signals...
Press Ctrl+C to send SIGINT.
Use another terminal to send SIGTERM or SIGUSR1.
^C
SIGINT received (Ctrl+C).
Handling SIGINT asynchronously...

SIGUSR1 received.
Handling SIGUSR1 asynchronously...

SIGTERM received.
Handling SIGTERM asynchronously...
Terminating program...
Program terminated successfully.
koumudi@Koumudi:~/OSSP/PRACTICAL/Session_6$ |
```